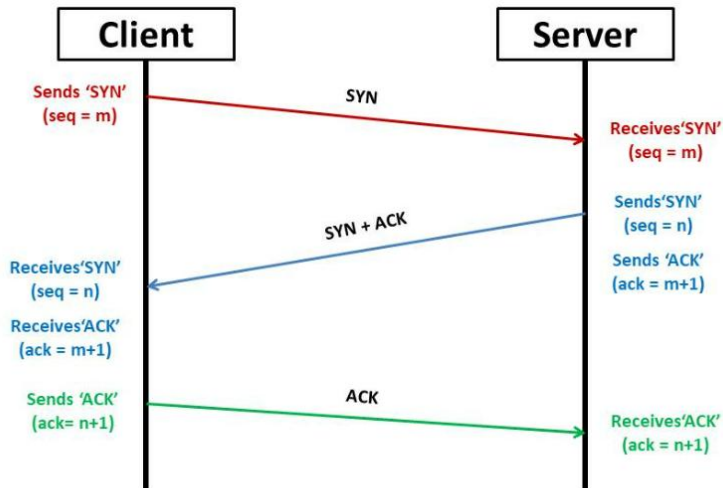


TCP vs UDP

At the transport layer (Layer 4), both TCP and UDP move data between two machines over IP.



3-Way Handshaking(for establishing connection)

Core Differences

TCP (Transmission Control Protocol)

- **Connection-oriented**: establishes a connection before sending data
- **Reliable**: guarantees delivery, order, and error-checking
- **Slower**: overhead from acknowledgments and retransmissions
- **Stream-based**: data is a continuous stream of bytes

UDP (User Datagram Protocol)

- **Connectionless**: just sends packets without establishing a connection
- **Unreliable**: no delivery guarantees, packets can be lost or arrive out of order
- **Faster**: minimal overhead, no retransmissions
- **Message-based**: sends discrete datagrams

When to Use Each

Use TCP when:

- Data integrity is critical (financial transactions, file transfers, databases)
- Order matters (chat messages, API requests/responses)
- You need confirmation of delivery
- Examples: HTTP/HTTPS, email (SMTP), SSH, database connections

Use UDP when:

- Speed is more important than reliability
- You can tolerate some data loss
- Real-time performance matters more than perfect accuracy
- Examples: video streaming, online gaming, VoIP, DNS lookups, live broadcasts

System Design Interview Scenarios

Video streaming (Netflix, YouTube)

- Use UDP or UDP-based protocols for live streaming
- Can use TCP for on-demand video where buffering is acceptable
- Tradeoff: occasional frame drops vs smooth playback

Multiplayer gaming

- Use UDP for player position updates (100+ updates/sec)
- Use TCP for critical events (purchasing items, chat)
- Reasoning: losing one position update is fine, the next one arrives in 10ms

Messaging app (WhatsApp, Slack)

- Use TCP for message delivery
- Need guarantee that "sent" messages actually arrive
- Order is critical for conversation context

Video conferencing (Zoom)

- Use UDP for audio/video streams
- Brief audio glitches are better than waiting for retransmission
- Can use TCP for screen sharing where accuracy matters

DNS

- Uses UDP by default (faster lookups)
- Falls back to TCP for large responses
- Single query-response, retries if needed at application layer

Key Interview Points

TCP guarantees come at a cost:

- Head-of-line blocking: if one packet is lost, subsequent packets wait
- Connection overhead: 3-way handshake adds latency
- Buffering and flow control add complexity

UDP requires application-level handling:

- You build your own reliability if needed (like QUIC does)
- Must handle packet loss, duplication, reordering yourself
- More control but more complexity

Hybrid approaches:

- QUIC (HTTP/3): UDP-based but adds reliability features
- Custom protocols: UDP for data, TCP for control
- WebRTC: UDP preferred, TCP fallback